Tell Claude to build a **small validated C++ prototype**, not “rewrite all of Warp Factory.” The scope has to be brutally controlled or it’ll wander into GUI nonsense and metric libraries.

Editar

You are helping me develop a research prototype for differentiable warp-spacetime analysis in C++.

The long-term goal is to create an independent C++ implementation of the essential numerical physics pipeline used by Warp Factory, with HyPAD-style hypercomplex automatic differentiation built in from the beginning for parameter sensitivities and gradient-based optimization.

Do not attempt to reproduce the full Warp Factory MATLAB project. Do not implement plotting, GUI features, animations, GPU support, or a large metric library.

Primary objective

Create a minimal, modular, testable C++ framework that computes:

$$
\mathbf{p}
\rightarrow
g_{\mu\nu}(\mathbf{x};\mathbf{p})
\rightarrow
\partial_\alpha g_{\mu\nu}
\rightarrow
\Gamma^\rho_{\mu\nu}
\rightarrow
R_{\mu\nu}
\rightarrow
R
\rightarrow
G_{\mu\nu}
\rightarrow
T_{\mu\nu}
\rightarrow
$$

J(\mathbf{p}),
]

where:

- (\mathbf{p}) is a small set of metric parameters;
- (g_{\mu\nu}) is evaluated on a regular Cartesian 3D grid;
- spatial derivatives are computed numerically;
- (J) is initially a simple scalar physical objective;
- all numerical physics functions must be templated on the scalar type so that the same implementation works with both `double` and a future HyPAD scalar type.

Core architecture requirement

Every important numerical routine must follow this pattern:

```
template <typename Scalar>
...
```

Do not hard-code `double` inside the geometry, metric, stress-energy, or objective kernels.

The intended usage is:

```
compute_stress_energy<double>(...);
compute_stress_energy<HyPADScalar>(...);
```

The C++ physics pipeline must not need separate sensitivity equations. Parameter derivatives should propagate automatically once the parameters and metric are evaluated using a HyPAD-compatible scalar type.

Initial scope

Implement only the following.

1. Grid and field infrastructure

Create:

- a regular Cartesian 3D grid;
- grid dimensions (N_x,N_y,N_z);
- coordinate bounds;
- uniform spacings (dx,dy,dz);

- storage for a (4\times4) metric tensor at every grid point;
- storage for rank-2, rank-3, and rank-4 tensor fields when required.

Prefer clear, contiguous storage and explicit indexing.

Do not overengineer a generic tensor library. Implement only the ranks and dimensions needed for four-dimensional spacetime calculations.

2. Scalar-generic tensor algebra

Implement:

- (4\times4) matrix multiplication;
- transpose if needed;
- determinant;
- inverse;
- contractions required by the GR equations;
- identity checks.

Avoid dependencies that prevent custom scalar types from being used.

A small dependency such as Eigen is acceptable only if it works with custom scalar types. Otherwise, implement the required (4\times4) operations directly.

3. Metrics

Implement two initial metrics.

Minkowski metric

\operatorname{diag}(-1,1,1,1).
]

This is the primary zero-curvature test.

One simple parametrized warp metric

Implement a basic Alcubierre-type metric using a smooth shape function and a small parameter set such as:

- bubble velocity (v);
- bubble radius (R);

- wall thickness or steepness (\sigma).

Keep the metric parametrization isolated from the geometry code.

Use a structure similar to:

```
template <typename Scalar>
struct AlcubierreParameters {
    Scalar velocity;
    Scalar radius;
    Scalar sigma;
};
```

4. Numerical derivatives

Implement reusable finite-difference operators for scalar and tensor fields.

Initially support:

- second-order centered differences in the interior;
- second-order one-sided differences at boundaries;
- first spatial derivatives;
- second spatial derivatives or repeated first derivatives as needed.

Document the stencil formulas and indexing.

The implementation should allow the derivative order or boundary scheme to be replaced later.

Assume the initial metric is stationary unless a time derivative is explicitly required. Clearly document which temporal derivatives are assumed to vanish.

5. General-relativity geometry

Implement the following quantities using consistent conventions:

Inverse metric

[
g^{\mu\nu}.
]

Christoffel symbols

\partial_\sigma g_{\mu\nu}
\right).
]

Ricci tensor

Use one clearly documented Riemann/Ricci convention.

Ricci scalar

[
R = g^{\mu\nu}R_{\mu\nu}.
]

Einstein tensor

R_{\mu\nu}

\frac{1}{2}g_{\mu\nu}R.
]

Stress-energy tensor

Initially use geometrized units:

\frac{1}{8\pi}G_{\mu\nu}.
]

Make the normalization configurable so that SI units can be added later.

6. Initial scalar objective

Implement one simple differentiable scalar objective based on the energy density.

For example, compute an Eulerian or coordinate-frame energy-density field and integrate a smooth negative-energy penalty over the domain.

Avoid a hard `min` or `max` if possible. Use a smooth negative-part approximation such as:

\frac{1}{2}
\left(
\sqrt{x^2+\epsilon^2}-x

\right).
]

Then define:

\sum_{\text{grid points}}
\operatorname{neg}_\epsilon(\rho)
,dV.
]

Clearly document that this is an initial numerical objective, not yet a complete physical assessment of the energy conditions.

7. HyPAD integration interface

Do not implement the full HyPAD library unless a compatible implementation already exists in the provided repository.

Instead:

- create a clean scalar interface;
- ensure all physics kernels compile with custom arithmetic types;
- add a lightweight test scalar type if useful;
- define where parameter seeding and derivative extraction will occur;
- isolate the HyPAD adapter in a `sensitivity/` module.

Provide an example API resembling:

```
auto parameters = seed_parameters(real_parameters);

auto objective =
    evaluate_objective<HyPADScalar>(
        parameters,
        grid,
        settings
    );

auto gradient = extract_gradient(objective);
```

If HyPAD source code is available, integrate it only after the ordinary `double` pipeline passes the core tests.

8. Tests

Tests are a first-class deliverable.

Implement the following test levels.

Algebra tests

Verify:

$$
g_{\mu\alpha}g^{\alpha\nu}
\approx
\delta_\mu^\nu.
$$

Finite-difference tests

Use analytic functions such as:

$$
f(x,y,z)=x^2+2y^2+3z^2
$$

and trigonometric functions to verify first and second derivatives.

Report convergence under grid refinement.

Minkowski tests

Verify numerically that:

$$
\Gamma^\rho_{\mu\nu}\approx0,
$$

$$
R_{\mu\nu}\approx0,
$$

$$
G_{\mu\nu}\approx0,
$$

$$
T_{\mu\nu}\approx0.
$$

]

Symmetry tests

Verify expected tensor symmetries, including:

[
g_{\mu\nu}=g_{\nu\mu},
]

[
R_{\mu\nu}=R_{\nu\mu},
]

within numerical tolerance.

Parameter sensitivity tests

Once the HyPAD scalar works, compare:

[
\frac{\partial J}{\partial p_i}
]

against centered finite differences:

[
\frac{J(p_i+h)-J(p_i-h)}{2h}.
]

Test at least:

- velocity;
- radius;
- wall steepness.

Use multiple finite-difference step sizes and report the relative error.

9. Configuration

Add a small JSON configuration interface with fields such as:

```
{
  "grid": {
    "nx": 41,
    "ny": 41,
```

```
    "nz": 41,
    "x_min": -10.0,
    "x_max": 10.0,
    "y_min": -10.0,
    "y_max": 10.0,
    "z_min": -10.0,
    "z_max": 10.0
  },
  "metric": {
    "type": "alcubierre",
    "velocity": 0.2,
    "radius": 3.0,
    "sigma": 2.0
  },
  "conventions": {
    "metric_signature": "-+++",
    "units": "geometrized",
    "riemann_convention": "documented_internal"
  },
  "derivatives": {
    "interior_order": 2,
    "boundary_scheme": "one_sided"
  },
  "objective": {
    "type": "smooth_negative_energy",
    "epsilon": 1e-8
  }
}
```

Keep the contract small and explicit.

10. Build system and repository structure

Use modern C++17 or newer and CMake.

Recommended structure:

```
warp_hypad/
├── CMakeLists.txt
├── include/
│   ├── core/
│   ├── geometry/
│   ├── metrics/
│   ├── physics/
```

```
│   ├── sensitivity/
│   └── optimization/
├── src/
├── tests/
├── examples/
├── config/
└── docs/
```

Use a unit-testing framework such as Catch2 or GoogleTest.

Required deliverables

Produce:

1. A repository architecture proposal.
2. A short technical design document.
3. The first compilable C++ implementation.
4. Unit tests.
5. One executable example for Minkowski spacetime.
6. One executable example for the parametrized Alcubierre metric.
7. A JSON-driven command-line executable.
8. Output summaries containing:
    - maximum absolute curvature values;
    - maximum stress-energy components;
    - integrated objective;
    - runtime;
    - grid information.
9. Documentation of all sign and index conventions.
10. A clear list of missing features and current limitations.

Development order

Work in this exact order:

Milestone 1

- repository skeleton;
- grid;
- fixed-size tensor structures;
- matrix inversion;
- algebra tests.

Milestone 2

- finite-difference operators;
- analytic derivative tests;
- grid-convergence output.

Milestone 3

- Minkowski metric;
- Christoffel symbols;
- Ricci tensor;
- Einstein tensor;
- Minkowski zero-curvature verification.

Milestone 4

- parametrized Alcubierre metric;
- stress-energy field;
- scalar objective.

Milestone 5

- generic scalar cleanup;
- custom scalar compatibility;
- HyPAD adapter.

Milestone 6

- gradient comparison against centered finite differences;
- simple gradient-descent demonstration.

Do not begin the next milestone until the previous milestone builds and its tests pass.

Important constraints

- Do not claim exact reproduction of Warp Factory yet.
- Do not copy MATLAB code line by line.
- Implement the equations independently and document them.
- Do not sacrifice clarity for optimization.
- Do not add CUDA, MPI, Python bindings, GUI features, or visualization in the first version.
- Do not optimize performance before numerical correctness is established.

- Avoid global state.
- Avoid hidden unit conventions.
- Avoid ambiguous tensor index ordering.
- Treat tests as part of the implementation, not as optional follow-up work.

First task

Start by inspecting the available repository and determining whether any existing HyPAD scalar implementation can be reused.

Then produce:

1. the proposed folder structure;
2. the scalar and tensor type design;
3. the grid-memory layout;
4. the exact index conventions;
5. the finite-difference strategy;
6. the GR sign conventions;
7. the Milestone 1 implementation;
8. the Milestone 1 tests.

Do not attempt the complete project in one unverified code dump. Build it incrementally and report which tests pass after each milestone.

One important tweak: give Claude your **HyPAD repository alongside this prompt**. Otherwise it may invent a fake `HyPADScalar` interface and create another glorious abstraction that connects to absolutely nothing.

you did not put like the visualizations in the milestones

You're right—I scoped the numerical engine but forgot that **the project also needs to produce interpretable spacetime maps and optimization plots**. Without visualizations, we would have numbers but no convincing way to inspect or present the physics.

Add this to the Claude prompt, replacing the milestone and deliverables sections:

Editar

Visualization and scientific-output requirements

Visualizations are a core deliverable, not optional postprocessing.

Do not build a GUI in the initial version. Instead, create reproducible plotting scripts using Python and Matplotlib. The C++ executable should export structured numerical results to files that Python can load without proprietary software.

Use one of the following output formats:

- HDF5 for full 3D tensor and scalar fields;
- CSV for convergence tables, optimization histories, and one-dimensional profiles;
- JSON for run metadata, parameters, conventions, and summary quantities;
- optionally VTK for inspection in ParaView.

Do not use MATLAB for visualization.

The visualization layer must remain separate from the C++ physics engine:

```
C++ solver
    ↓
HDF5 / CSV / JSON / VTK
    ↓
Python plotting scripts
    ↓
PNG / PDF / SVG figures
```

All plots must include:

- axis labels;
- units or a clear statement that geometrized units are used;
- parameter values;
- grid resolution;
- metric name;
- quantity being shown;
- slice location;

- color bar where applicable;
- descriptive file names.

Use consistent coordinate orientation across all plots.

Required initial visualizations

1. Metric shape function

For the Alcubierre-type metric, plot the shape function through the bubble center along the direction of motion.

Plot:

$$
f(r)
$$

as a function of position for multiple values of:

- bubble radius (R);
- wall steepness (\sigma).

This plot should confirm that the parameterized metric behaves as expected before curvature is calculated.

2. Two-dimensional metric-component slices

Produce central-plane slices for selected nontrivial metric components, for example:

$$
g_{00},\qquad g_{0x},\qquad g_{xx}.
$$

At minimum, support:

- (x)-(y) slice at the central (z);
- (x)-(z) slice at the central (y);
- (y)-(z) slice at the central (x).

Only plot components that are informative for the selected metric.

3. Energy-density slices

Plot the selected energy-density field on central planes.

Initially produce:

- a signed energy-density map;
- a negative-energy-only map;
- a one-dimensional profile through the bubble center.

The signed visualization must use a color scale centered at zero whenever positive and negative values are both present.

4. Stress-energy component slices

Allow plotting selected components such as:

$$
T_{00},\qquad T_{0x},\qquad T_{xx}.
$$

The plotting script should accept the component indices from the command line or configuration file.

5. Curvature visualizations

Plot central slices of quantities such as:

$$
R,
\qquad
R_{\mu\nu}R^{\mu\nu},
\qquad
G_{00}.
$$

At minimum, implement the Ricci scalar and one additional curvature or Einstein-tensor diagnostic.

6. Negative-energy volume visualization

Produce a mask showing the region where the selected energy density is negative:

$$
\rho < 0.
$$

Support:

- two-dimensional slice visualization;
- optional export of the mask to VTK for 3D viewing in ParaView.

Do not make a custom 3D renderer in the first version.

7. Sensitivity maps

Once HyPAD is integrated, export and plot spatial sensitivity fields such as:

$$
\frac{\partial \rho(\mathbf{x})}{\partial v},
\qquad
\frac{\partial \rho(\mathbf{x})}{\partial R},
\qquad
\frac{\partial \rho(\mathbf{x})}{\partial \sigma}.
$$

Also support sensitivity maps for selected stress-energy components:

$$
\frac{\partial T_{\mu\nu}(\mathbf{x})}{\partial p_i}.
$$

These are central scientific outputs of the project.

The sensitivity plotting script must clearly indicate:

- the differentiated output;
- the selected parameter;
- the parameter value;
- whether the derivative was obtained with HyPAD or finite differences.

8. HyPAD-versus-finite-difference comparison

For each tested parameter, create a comparison figure showing:

- HyPAD derivative;
- centered finite-difference derivative;
- absolute difference;
- relative difference where numerically meaningful.

For spatial fields, generate separate slice plots for:

$$
\left.\frac{\partial \rho}{\partial p_i}\right|_{\mathrm{HyPAD}},
\qquad
\left.\frac{\partial \rho}{\partial p_i}\right|_{\mathrm{FD}},
\qquad
\Delta.
$$

Also generate a summary table containing:

- parameter;
- HyPAD derivative of the scalar objective;
- finite-difference derivative;
- absolute error;
- relative error;
- finite-difference step size.

9. Grid-convergence plots

For the finite-difference and GR tests, plot numerical error versus grid spacing or grid resolution.

Use log-log plots where appropriate.

Include:

- first-derivative error;
- second-derivative error;
- Minkowski residual magnitude;
- known-solution curvature error if an analytic curved metric is added;
- estimated observed convergence order.

10. Optimization-history plots

Once gradient-based optimization is implemented, plot:

- objective value versus iteration;
- gradient norm versus iteration;
- each metric parameter versus iteration;
- constraint values versus iteration;
- runtime or cumulative function evaluations versus iteration.

Export the full optimization history to CSV.

11. Before-and-after optimization comparison

Generate comparable figures for the initial and optimized metrics:

- metric shape function;
- energy-density slice;
- negative-energy region;
- curvature diagnostic;
- integrated objective;
- selected physical constraints.

Use identical grid extents and color limits where possible so the comparison is not visually misleading.

12. Performance and scaling plots

Once ordinary and HyPAD evaluations are available, measure and plot:

- runtime versus grid size;
- memory versus grid size;
- runtime versus number of active parameters;
- memory versus number of active parameters;
- HyPAD runtime relative to ordinary `double`;
- HyPAD gradient cost relative to finite differences.

These plots are required for evaluating whether HyPAD improves parameter-optimization workflows.

Output file organization

Use a run-specific output directory:

```
results/
└── <run_name>/
    ├── metadata.json
    ├── summary.json
    ├── fields.h5
    ├── profiles/
    ├── convergence/
    ├── sensitivities/
    ├── optimization/
    └── figures/
```

The metadata must record:

- git commit;
- build type;
- compiler;
- grid settings;
- metric parameters;
- physical conventions;
- derivative settings;
- HyPAD settings;
- objective definition;
- run timestamp.

Every figure must be reproducible from saved output files without rerunning the C++ solver.

Revised required deliverables

Produce:

1. Repository architecture proposal.
2. Short technical design document.
3. First compilable C++ implementation.
4. Unit and convergence tests.
5. Minkowski executable example.
6. Parametrized Alcubierre executable example.
7. JSON-driven command-line executable.
8. HDF5, CSV, JSON, or VTK result export.
9. Python visualization package.
10. Metric shape-function plots.
11. Metric-component slice plots.
12. Energy-density and stress-energy maps.
13. Curvature diagnostic maps.
14. Negative-energy masks.
15. HyPAD sensitivity maps.
16. HyPAD-versus-finite-difference comparison figures.
17. Grid-convergence plots.
18. Optimization-history plots.
19. Before-and-after optimization figures.
20. Runtime and memory scaling plots.
21. Documentation of sign, index, frame, and unit conventions.
22. Clear list of missing features and limitations.

Revised development milestones

Milestone 1 — Core data structures

Implement:

- repository skeleton;
- regular Cartesian grid;
- fixed-size tensor structures;
- contiguous field storage;
- matrix inversion;
- configuration parsing;
- algebra tests.

Required output:

- passing algebra tests;
- documented index and memory layout;
- sample JSON configuration;
- metadata-export prototype.

Milestone 2 — Numerical derivatives and convergence visualization

Implement:

- first spatial derivatives;
- second spatial derivatives;
- centered interior stencils;
- one-sided boundary stencils;
- analytic derivative tests;
- grid-refinement studies.

Required visualizations:

- analytic versus numerical derivative profiles;
- error versus grid resolution;
- log-log convergence plots;
- observed convergence-order table.

Do not continue until the expected convergence order is demonstrated.

Milestone 3 — Minkowski spacetime and zero-field visualization

Implement:

- Minkowski metric;
- inverse metric;
- Christoffel symbols;
- Ricci tensor;
- Ricci scalar;
- Einstein tensor;
- stress-energy tensor.

Required visualizations:

- central slices of the largest computed Christoffel component;
- Ricci scalar slice;
- (G_{00}) slice;
- (T_{00}) slice;
- residual magnitude versus grid resolution.

These plots should show numerical zero within tolerance and expose boundary artifacts if present.

Milestone 4 — Parametrized Alcubierre metric

Implement:

- smooth bubble shape function;
- velocity (v);
- radius (R);
- steepness (\sigma);
- metric field generation;
- stress-energy calculation;
- initial scalar objective.

Required visualizations:

- shape-function profiles;
- selected metric-component slices;
- energy-density slices;
- stress-energy component slices;
- Ricci scalar or curvature diagnostic;
- negative-energy mask;
- centerline profiles.

Milestone 5 — Field export and reusable plotting package

Implement:

- HDF5 or equivalent field export;
- CSV summary export;
- JSON metadata;
- optional VTK export;
- Python command-line plotting scripts.

Required outcome:

```
python plot_slice.py results/run_001 --field energy_density
```

should generate a publication-quality figure without rerunning the solver.

Add plotting regression or smoke tests to ensure scripts run successfully.

Milestone 6 — Generic scalar and HyPAD integration

Implement:

- scalar-generic cleanup;
- custom scalar compatibility;
- HyPAD adapter;
- parameter seeding;
- derivative extraction;
- sensitivity-field export.

Required visualizations:

- (\partial \rho/\partial v);
- (\partial \rho/\partial R);
- (\partial \rho/\partial \sigma);
- sensitivities of selected (T_{\mu\nu}) components;
- sensitivity centerline profiles.

Milestone 7 — HyPAD verification

Implement:

- centered finite-difference parameter derivatives;

- multiple finite-difference step sizes;
- scalar-objective comparison;
- full-field sensitivity comparison.

Required visualizations:

- HyPAD versus finite-difference derivative plots;
- absolute-error maps;
- relative-error maps;
- error versus finite-difference step size;
- derivative verification summary table.

Do not claim sensitivity correctness until these comparisons pass defined tolerances.

Milestone 8 — Gradient-based optimization

Implement:

- bounded parameter optimization;
- simple gradient descent or L-BFGS;
- parameter bounds;
- objective and gradient logging;
- basic constraint handling.

Required visualizations:

- objective versus iteration;
- gradient norm versus iteration;
- parameters versus iteration;
- initial versus optimized energy-density maps;
- initial versus optimized negative-energy masks;
- initial versus optimized curvature maps;
- initial versus optimized shape-function profiles.

Milestone 9 — Performance and scaling study

Benchmark:

- ordinary `double`;
- HyPAD;
- centered finite differences;
- increasing grid resolutions;
- increasing active-parameter counts.

Required visualizations:

- runtime versus grid size;
- peak memory versus grid size;
- runtime versus number of parameters;
- memory versus number of parameters;
- total gradient cost by method;
- objective reduction versus total computational cost.

Milestone 10 — Warp Factory cross-validation

This milestone may be completed later when MATLAB and Warp Factory are available.

Implement comparison cases using:

- identical metric parameters;
- identical domain;
- identical grid;
- matching sign conventions;
- matching finite-difference settings where possible;
- matching physical normalization.

Required visualizations:

- C++ versus Warp Factory energy-density slices;
- difference maps;
- centerline comparisons;
- stress-energy component comparisons;
- integrated-objective comparison;
- error summary table.

Do not block Milestones 1–9 on MATLAB availability. Clearly mark Warp Factory cross-validation as pending until it is completed.

First task

Begin with Milestone 1 only.

Before implementing it, inspect the available repository and identify whether an existing HyPAD scalar implementation can be reused.

Then produce:

1. proposed folder structure;

2. scalar and tensor design;
3. grid-memory layout;
4. index conventions;
5. sign and curvature conventions;
6. output and visualization data format;
7. Milestone 1 implementation;
8. Milestone 1 tests;
9. a concise statement of which Milestone 1 requirements pass.

Do not jump ahead and generate placeholder visualizations using fake physics data.

That gives the project a proper visual progression: first numerical convergence, then spacetime fields, then **HyPAD sensitivity maps**, then optimization before/after comparisons, and finally Warp Factory difference maps.

put verifications for everything

Absolutely. Every milestone should have a **verification gate**, not just "it runs." Claude should be forced to prove each component independently before building on top of it.

Add this section to the prompt and use it to replace the current milestone plan:

Editar

Verification-first development policy

Every implemented feature must include:

1. a mathematical or numerical reference;
2. an automated test;
3. a quantitative acceptance tolerance;
4. a saved verification report;
5. a visualization where visual inspection adds useful information;
6. a failure condition that blocks later milestones.

A milestone is not complete merely because the code compiles or produces plausible figures.

For every verification case, record:

- test name;
- mathematical reference or expected behavior;
- grid resolution;
- parameters;
- expected result;
- computed result;
- absolute error;
- relative error;
- tolerance;
- pass or fail;
- runtime;
- git commit;
- compiler and build mode.

Store verification summaries in:

```
results/
└── verification/
    ├── milestone_01/
    ├── milestone_02/
    ├── milestone_03/
    └── ...
```

Each milestone must produce:

```
verification_summary.json
verification_table.csv
verification_report.md
```

No milestone may be marked complete while any required verification test is failing.

General verification hierarchy

Use the following hierarchy whenever possible:

Level A — Exact algebraic identities

Compare against quantities that must be exactly zero, one, symmetric, or invariant.

Level B — Analytic solutions

Compare numerical outputs against closed-form derivatives, metrics, curvature tensors, or parameter sensitivities.

Level C — Manufactured solutions

Construct fields with known derivatives and insert them into isolated numerical operators.

Level D — Independent numerical methods

Compare HyPAD against centered finite differences, complex-step differentiation where applicable, or another independently written implementation.

Level E — Grid-convergence studies

Demonstrate that error decreases at the expected order as the grid is refined.

Level F — Cross-code validation

Later compare selected results against Warp Factory using matched inputs and conventions.

Do not use one implementation path to verify itself. Verification calculations must be independent wherever practical.

Revised milestones with mandatory verification

Milestone 1 — Grid, storage, tensors, and matrix algebra

Implement:

- repository skeleton;
- Cartesian grid;
- coordinate generation;
- tensor indexing;
- contiguous field storage;
- fixed-size (4\times4) matrix operations;
- determinant;

- inverse;
- contractions;
- JSON configuration;
- metadata export.

Required verifications

Grid coordinates

Verify:

- first coordinate equals the lower bound;
- final coordinate equals the upper bound;
- spacing is uniform;
- center coordinate is correct for odd grid dimensions;
- flattening and unflattening indices are mutually consistent.

For every point:

[
i_{\mathrm{flat}}
\rightarrow
(i,j,k)
\rightarrow
i_{\mathrm{flat}}
]

must return the original flat index.

Memory layout

Create a known synthetic tensor field where every component encodes its indices, for example:

10^6 i + 10^4 j + 10^2 k + 10\mu+\nu.
]

Verify that every stored and retrieved value matches exactly.

Matrix multiplication

Compare against hand-calculated matrices and randomly generated matrices evaluated by an independent reference implementation.

Matrix inversion

Verify:

$$
A A^{-1} \approx I, \qquad A^{-1}A \approx I.
$$

Use:

- diagonal matrices;
- symmetric matrices;
- Lorentzian metric-like matrices;
- randomly generated nonsingular matrices.

Test singular and nearly singular matrices and verify that the code reports failure or poor conditioning rather than silently returning nonsense.

Determinant

Compare against known analytic determinants and an independent reference implementation.

Tensor contractions

Verify contractions using small hand-calculated tensors with known results.

Scalar genericity

Compile and run tensor operations using:

- `double`;
- `long double`;
- a lightweight custom scalar wrapper.

This confirms that the core does not secretly depend on `double`.

Required visualizations

- diagram or table documenting memory layout;

- matrix-inverse residual histogram;
- condition number versus inversion residual, if condition estimates are implemented.

Acceptance gate

Milestone 1 passes only if:

- all exact indexing tests pass;
- matrix identity residuals remain below a defined tolerance;
- singular matrices are detected;
- all core operations compile with the test scalar type;
- configuration round-trip tests pass.

Milestone 2 — Finite-difference operators

Implement:

- first derivatives;
- second derivatives;
- mixed derivatives if required;
- centered interior stencils;
- one-sided boundary stencils;
- tensor-field differentiation.

Required verifications

Use multiple analytic functions.

Polynomial fields

For example:

$$x^4 + 2y^3 - 3z^2 + xyz.$$
]

Derive and compare:

$$[
\frac{\partial f}{\partial x},
\quad
\frac{\partial f}{\partial y},
\quad$$

\frac{\partial f}{\partial z},
]

[
\frac{\partial^2 f}{\partial x^2},
\quad
\frac{\partial^2 f}{\partial y^2},
\quad
\frac{\partial^2 f}{\partial z^2},
]

and mixed derivatives such as:

[
\frac{\partial^2 f}{\partial x\partial y}.
]

Trigonometric fields

Use a smooth periodic field such as:

\sin(ax)\cos(by)e^{cz}.
]

Compare all implemented derivatives against analytic expressions.

Constant and linear fields

Verify:

- derivative of a constant is zero;
- first derivative of a linear field is exact within roundoff;
- second derivative of a linear field is zero.

Tensor fields

Construct a tensor field whose components use different analytic functions and verify every differentiated component independently.

Boundary verification

Evaluate interior and boundary errors separately.

Report:

- maximum interior error;

- maximum boundary error;
- L_1, L_2, and L_∞ errors.

Grid convergence

Use at least four grid resolutions.

Estimate observed order:

$$\frac{\log(E_h/E_{h/2})}{\log 2}.$$

The observed order must approach the theoretical stencil order in the asymptotic regime.

Commutativity of mixed derivatives

For smooth fields verify:

$$\frac{\partial^2 f}{\partial x\partial y} \approx \frac{\partial^2 f}{\partial y\partial x}.$$

Required visualizations

- analytic versus numerical derivative profiles;
- pointwise error slices;
- boundary versus interior error;
- log-log error versus grid spacing;
- observed convergence order versus resolution.

Acceptance gate

Milestone 2 passes only if:

- every derivative operator passes analytic tests;
- boundary behavior is quantified;
- observed convergence order agrees with the expected order within a specified margin;
- tensor differentiation matches componentwise analytic results;
- mixed derivatives commute within tolerance.

Milestone 3 — Metric generation

Implement:

- Minkowski metric;
- parametrized Alcubierre-type metric;
- metric symmetry enforcement or checking;
- metric determinant;
- inverse metric;
- shape function.

Required verifications

Minkowski metric

At every grid point verify:

\operatorname{diag}(-1,1,1,1).
]

Verify:

g_{\mu\nu},
\qquad
\det g=-1.
]

Metric symmetry

Verify:

[
g_{\mu\nu}=g_{\nu\mu}
]

at every grid point.

Inverse relation

Verify:

\delta_\mu^\nu
]

for every point of every implemented metric.

Lorentzian signature

Check the eigenvalue signature or an equivalent robust criterion.

Verify that each valid metric point has one timelike and three spacelike directions under the chosen convention.

Flag any parameter combination that creates:

- singular metrics;
- incorrect signature;
- non-finite values.

Shape-function limits

For the warp metric verify:

- appropriate interior value;
- appropriate exterior value;
- smooth transition through the wall;
- monotonic behavior where expected;
- asymptotic approach to flat spacetime;
- expected effects of changing (R) and (\sigma).

Parameter perturbation behavior

Verify independently that:

- increasing (R) shifts the wall location;
- increasing (\sigma) changes wall steepness;
- changing (v) changes only the expected metric components.

Required visualizations

- shape-function profiles for multiple parameter values;
- determinant slices;
- minimum absolute determinant over the domain;
- signature-failure mask;
- selected metric-component slices;
- inverse-identity residual map.

Acceptance gate

Milestone 3 passes only if:

- all metrics are symmetric;
- inverse residuals pass tolerance everywhere;

- valid test cases preserve Lorentzian signature;
- Minkowski values are exact within roundoff;
- parameter changes produce the mathematically expected behavior.

Milestone 4 — Christoffel symbols

Implement:

\partial_\sigma g_{\mu\nu}
\right).
]

Required verifications

Lower-index symmetry

For a Levi-Civita connection verify:

\Gamma^\rho_{\nu\mu}.
]

Minkowski test

Verify:

[
\Gamma^\rho_{\mu\nu}=0
]

throughout the domain.

Analytic metric test

Add at least one nontrivial metric with known Christoffel symbols.

A simple curvilinear-coordinate flat metric may be used if Cartesian-only assumptions are clearly isolated, or use another stationary metric with analytically available components.

Compare selected Christoffel components against analytic values.

Metric compatibility

Numerically verify:

\partial_\lambda g_{\mu\nu}

\Gamma^\sigma_{\lambda\mu}g_{\sigma\nu}

\Gamma^\sigma_{\lambda\nu}g_{\mu\sigma}
\approx0.
]

This is a critical independent identity.

Required visualizations

- largest Christoffel-component slice;
- symmetry residual map;
- metric-compatibility residual map;
- analytic versus numerical profiles for selected components;
- convergence of Christoffel error with grid refinement.

Acceptance gate

Milestone 4 passes only if:

- lower-index symmetry holds;
- Minkowski connection is numerically zero;
- analytic component comparisons pass;
- metric compatibility converges toward zero at the expected order.

Milestone 5 — Riemann, Ricci, and scalar curvature

Implement:

- Riemann tensor or the contractions needed to obtain Ricci;
- Ricci tensor;
- Ricci scalar;
- optional curvature invariants.

Document the exact sign convention.

Required verifications

Minkowski curvature

Verify:

[
R^\rho{}{\sigma\mu\nu}=0,
\quad
R{\mu\nu}=0,
\quad
R=0.
]

Ricci symmetry

Verify:

[
R_{\mu\nu}=R_{\nu\mu}.
]

Riemann antisymmetries

As applicable to the stored index arrangement, verify identities such as:

-R_{\sigma\rho\mu\nu},
]

-R_{\rho\sigma\nu\mu}.
]

Pair symmetry

Verify:

R_{\mu\nu\rho\sigma}
]

after lowering indices, within numerical tolerance.

First Bianchi identity

Verify:

[
R_{\rho[\sigma\mu\nu]}
\approx0.
]

Analytic curved metric

Add at least one metric with known curvature.

Suitable choices include:

- de Sitter spacetime;
- anti-de Sitter spacetime;
- a spatially flat FLRW metric at a fixed time if temporal derivatives are supported;
- a static constant-curvature metric compatible with the implementation assumptions.

Compare:

- selected Riemann components;
- Ricci tensor;
- Ricci scalar;
- one invariant.

Coordinate-flat test

Where practical, include a flat spacetime represented in nontrivial coordinates to verify that nonzero Christoffel symbols can still produce zero curvature.

Grid convergence

Demonstrate expected convergence for analytic curvature quantities.

Required visualizations

- Ricci scalar slices;
- analytic versus numerical curvature profiles;
- Riemann-symmetry residual maps;
- Bianchi residual maps;
- curvature error versus grid spacing;
- observed convergence order.

Acceptance gate

Milestone 5 passes only if:

- Minkowski curvature is zero;

- Ricci symmetry passes;
- implemented Riemann identities pass;
- analytic curved-metric errors converge at the expected rate;
- the chosen sign convention matches the documented analytic reference.

Milestone 6 — Einstein tensor and stress-energy tensor

Implement:

R_{\mu\nu}

\frac{1}{2}g_{\mu\nu}R,
]

\frac{1}{8\pi}G_{\mu\nu}
]

in geometrized units.

Required verifications

Einstein tensor symmetry

Verify:

[
G_{\mu\nu}=G_{\nu\mu}.
]

Minkowski test

Verify:

[
G_{\mu\nu}=0,
\qquad
T_{\mu\nu}=0.
]

Trace identity

In four dimensions verify:

$$
G^\mu{}_\mu=-R.
$$

Contracted Bianchi identity

Numerically verify:

$$
\nabla_\mu G^{\mu\nu}\approx0.
$$

This must be reported separately for:

- interior points;
- boundary points;
- each value of ν.

Stress-energy normalization

Verify that changing the normalization constant produces the expected exact scaling.

Known matter or cosmological metric

For an analytic reference metric with known stress-energy or cosmological constant, compare the computed Einstein tensor against the expected form.

For example, for a constant-curvature spacetime verify proportionality:

$$
G_{\mu\nu}\propto g_{\mu\nu}.
$$

Required visualizations

- G_{00} and selected component slices;
- T_{00} and selected component slices;
- divergence-of-G residual maps;
- trace-identity error maps;
- error convergence plots.

Acceptance gate

Milestone 6 passes only if:

- symmetry and trace identities pass;
- Minkowski gives zero;
- the contracted Bianchi residual decreases with refinement;
- analytic reference comparisons pass;
- normalization scaling is correct.

Milestone 7 — Observer frames and energy density

Implement a clearly defined observer field and calculate:

T_{\mu\nu}u^\mu u^\nu.
]

Do not call (T_{00}) “energy density” unless that identification is valid for the chosen frame.

Required verifications

Four-velocity normalization

Verify:

[
g_{\mu\nu}u^\mu u^\nu=-1.
]

Frame-basis orthonormality

If a tetrad is used, verify:

\eta_{\hat a\hat b}.
]

Tensor transformation consistency

Transform a tensor into the observer frame and back.

Verify recovery of the original tensor within tolerance.

Minkowski observer test

For simple synthetic stress-energy tensors in Minkowski spacetime, compare the computed observer energy density against hand-calculated values.

Lorentz transformation test

Use a known boost and verify expected transformation behavior for a simple stress-energy tensor.

Multiple observer consistency

Test that equivalent observer definitions produce identical energy densities.

Required visualizations

- observer normalization residual;
- orthonormal-frame residual maps;
- energy-density slices;
- comparison between coordinate (T_{00}) and observer-measured (\rho);
- centerline profiles.

Acceptance gate

Milestone 7 passes only if:

- observer vectors are normalized;
- frames are orthonormal;
- round-trip transformations pass;
- analytic Minkowski observer tests pass;
- energy-density definitions are unambiguous and documented.

Milestone 8 — Energy conditions and scalar objectives

Implement selected diagnostics such as:

- weak energy condition;
- null energy condition;
- dominant energy condition;
- smooth negative-energy objective.

Required verifications

Synthetic tensors

Construct stress-energy tensors with known behavior:

- positive-energy dust;
- cosmological constant;
- electromagnetic-like tensor;
- explicit NEC-violating tensor;
- explicit WEC-violating tensor.

Verify that each energy-condition test returns the expected classification.

Observer-direction sampling

If observer or null-direction sampling is used, verify convergence as the number of sampled directions increases.

Document that finite sampling is approximate.

Algebraic alternatives

Where possible compare directional sampling against an independent eigenvalue-based or optimization-based check.

Smooth negative-part function

For:

$$
\frac{1}{2}
\left(
\sqrt{x^2+\epsilon^2}-x
\right),
$$

verify:

- non-negativity;
- smoothness;
- asymptotic agreement with $\max(0,-x)$;
- analytic first derivative;
- analytic second derivative;
- behavior as $\epsilon\rightarrow0$.

Numerical integration

Verify volume integration using constant, polynomial, and separable analytic fields with known integrals.

Objective decomposition

Verify that the global objective equals the sum of stored cellwise contributions.

Required visualizations

- energy-condition violation masks;
- sampling convergence versus number of directions;
- smooth penalty versus exact nonsmooth penalty;
- numerical integration error versus grid resolution;
- objective contribution maps.

Acceptance gate

Milestone 8 passes only if:

- synthetic classification tests pass;
- integration converges correctly;
- the smooth penalty matches its analytic derivatives;
- sampling limitations are quantified;
- objective reconstruction is exact within roundoff.

Milestone 9 — Data export and visualization pipeline

Implement:

- HDF5 or equivalent field export;
- CSV tables;
- JSON metadata;
- optional VTK;
- Python plotting scripts.

Required verifications

Round-trip field export

Generate a known synthetic field, write it, read it back, and verify exact or tolerance-level equality.

Test:

- scalar fields;
- tensor fields;
- sensitivity fields;
- coordinates;
- metadata.

Metadata completeness

Verify that required metadata fields are present:

- conventions;
- units;
- parameters;
- grid;
- git commit;
- compiler;
- build type;
- objective definition.

Figure reproducibility

Generate each figure twice from the same saved data.

Verify that numerical data underlying the plots is unchanged.

Orientation tests

Export a field with a clearly identifiable directional pattern.

Verify that:

- `xy`;
- `xz`;
- `yz`

plots use the intended axes and slice locations.

Color-scale tests

Verify that signed fields use zero-centered scales when required and that before-and-after comparisons use shared limits.

Command-line smoke tests

Run all plotting scripts in automated tests and verify that expected files are created and nonempty.

Required visualizations

Use synthetic orientation fields before plotting physics results.

Acceptance gate

Milestone 9 passes only if:

- file round trips pass;
- axes and orientations are correct;
- required metadata is complete;
- every required plot script executes automatically;
- no figure depends on manually edited intermediate files.

Milestone 10 — Generic scalar and HyPAD integration

Implement:

- custom HyPAD-compatible scalar;
- parameter seeding;
- derivative propagation;
- derivative extraction;
- spatial sensitivity fields.

Required verifications

Elementary arithmetic

Verify derivatives of:

- addition;
- subtraction;
- multiplication;
- division;
- reciprocal;
- square root;
- exponential;

- logarithm;
- hyperbolic functions;
- powers used in the metric.

Compare against analytic derivatives.

Composite scalar functions

Test functions with known gradients and Hessians.

For example:

a^2\sin b + e^{ac}.
]

Verify:

[
\frac{\partial f}{\partial a},
\quad
\frac{\partial f}{\partial b},
\quad
\frac{\partial f}{\partial c}.
]

If second-order HyPAD is supported, verify the Hessian.

Seed isolation

Seed one parameter at a time and verify that unrelated derivative channels remain zero.

Multiple simultaneous seeds

Verify that simultaneous parameter seeding produces the same derivatives as separate runs.

Scalar-generic equivalence

With all derivative seeds set to zero, verify that HyPAD real components match the `double` solution.

Compare every major field:

- metric;
- inverse metric;